

AMSS 2026 — Lecture 6: Behavioral I — Use Cases + Sequence

Traian-Florin Șerbănuță

2026

Today's Agenda

1. Frame: from structure to behaviour
2. Use cases: from concept to notation
3. Sequence diagrams realise use cases
4. Live demo: drive AI to a sequence
5. Reading critically: where AI fabricates messages
6. Bridge to Week 7 + Lab 3 this week

Recap: Architect and Critic

- ▶ **Architect / director:** drive AI through the software development lifecycle (SDLC).
- ▶ **Critic / reviewer:** read AI's output and name what's wrong or missing.

In Weeks 4-5 you critiqued AI-drawn *structure*. Today: the same loop, on *behaviour*.

From Structure to Behaviour

- ▶ **Weeks 4-5 pinned the parts** — classes, packages, components: *what the system is made of*.
- ▶ **Today: how the parts interact** — which object sends which message to which, and in what order.

Today's question: **does this interaction really happen — or did AI invent it?**

The Literacy Floor: Critique, Rationale, Traceability

From Week 1: in the oral defense, *unaided*, you must demonstrate:

- ▶ **Critique** — read & critique AI-generated artifacts on the spot.
- ▶ **Rationale** — articulate why you directed AI a certain way.
- ▶ **Traceability** — defend the trace across your project.

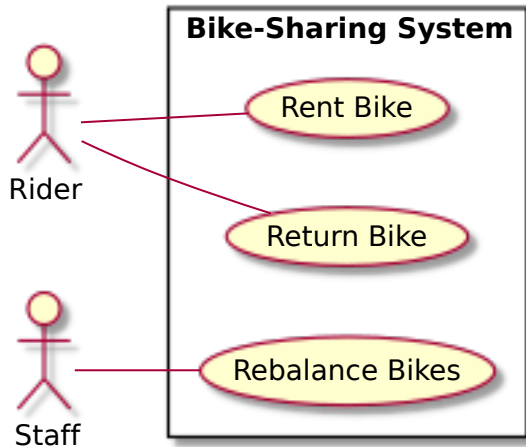
Today is Critique on behaviour: read an AI sequence and name the fabricated or missing messages. **Traceability touch:** requirement -> use case -> sequence -> class is the behavioural tail of the trace.

Use Cases: From Concept to Notation

- ▶ **Week 2 gave you the concept** — a use case is one thing a user wants to accomplish; a bucket for requirements.
- ▶ **Today: the notation** — actors, ovals, system boundary, and the relationships between use cases.

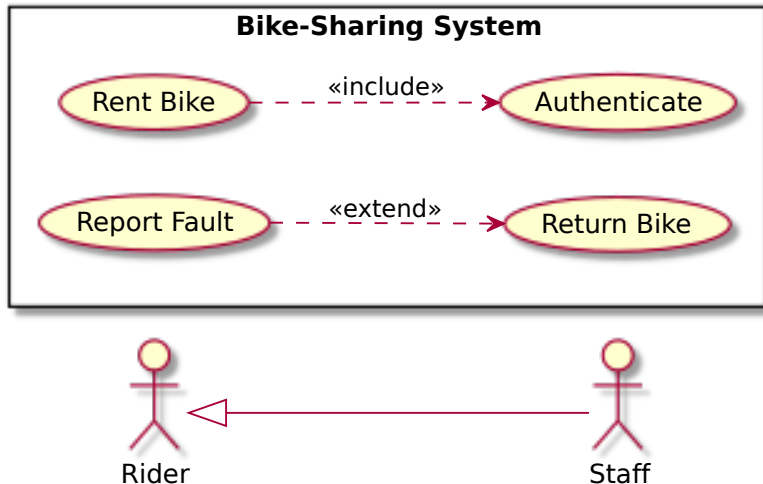
The use case says **what** the user wants; the sequence will say **how** the system delivers it.

Use Case Diagram



Actors (stick figures) sit outside; **use cases** (ovals) sit inside the **system boundary** (the box).

Include / Extend / Generalization



- ▶ **Include** — mandatory sub-behaviour: Rent Bike *always* includes Authenticate.
- ▶ **Extend** — optional: Report Fault may extend Return Bike.

Sequence Diagrams Realise Use Cases

A use case names a goal. A **sequence diagram** shows the objects collaborating to fulfil it — message by message, in time order.

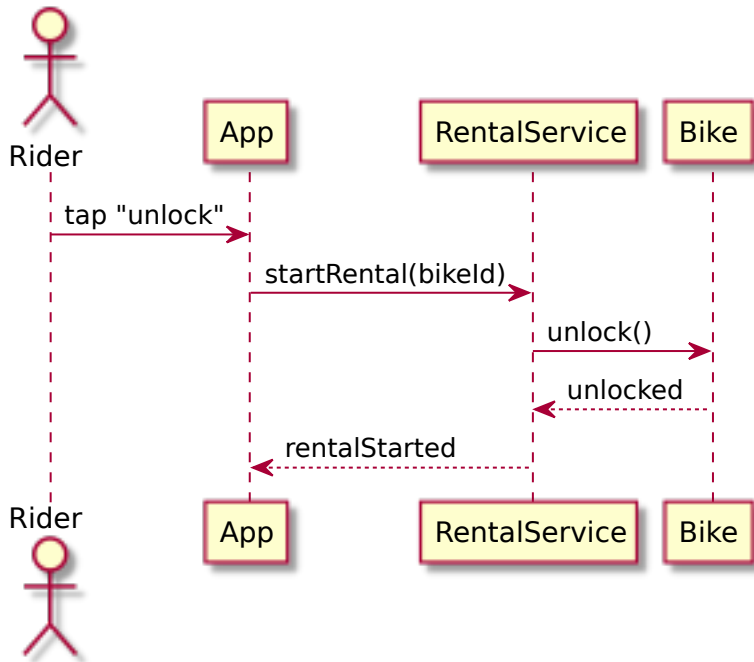
“Rent a Bike” (the use case) -> a sequence of messages between Rider, App, RentalService, Bike.

Demo: Drive AI to a Sequence

*Live: ask AI to draw the interaction for “rent a bike.”
Watch the messages — are they real, in the right order,
with the failure path?*

Prompt to AI: *“Generate a UML sequence diagram (as PlantUML) for renting a bike in the city bike-sharing app: a rider unlocks a bike at a station and is charged by app.”*

Sequence: Lifelines + Messages



Reading a Sequence

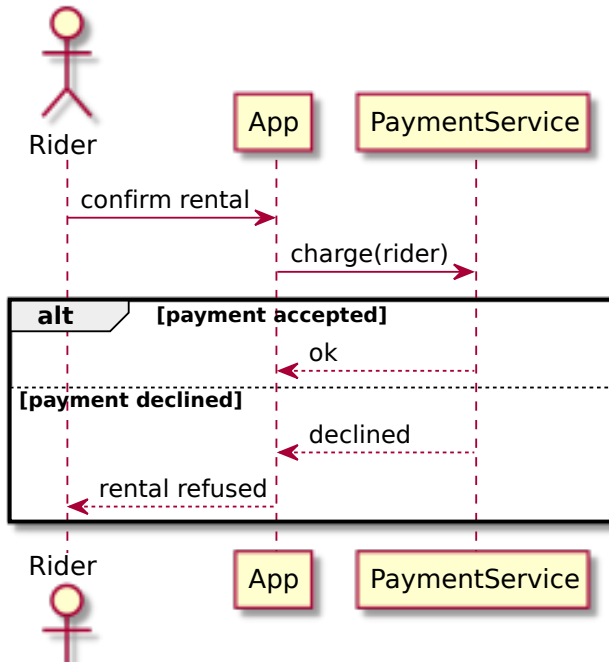
Read top to bottom, and check:

1. **Order** — does each message only depend on what happened above it?
2. **Returns** — does every call that needs an answer get a dashed return?
3. **Activation** — is it clear which object is executing at each step?

Your Turn: Spot the Break

Look back at the lifelines-and-messages sequence — but imagine
RentalService --> App : rentalStarted is sent *before* App
-> RentalService : startRental(bikeId).
With your neighbour, 90 seconds: which of the three checks
*— **order** / **returns** / **activation** — does this sequence*
break?

Alternative Paths

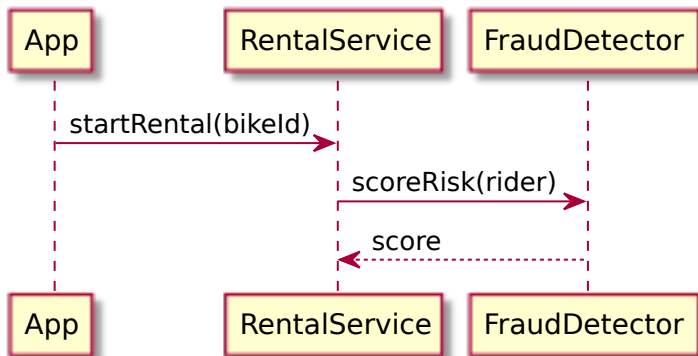


AI Draws Plausible Conversations — and Invents Messages

A sequence can read smoothly and still contain messages that never happen. The critic question:

“Does each message map to a real responsibility — or did AI invent it to connect the boxes?”

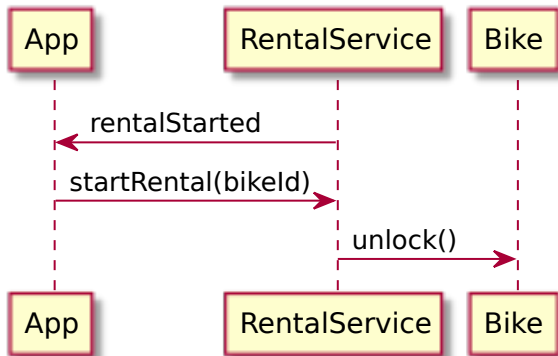
Defect #1: Fabricated Message



`FraudDetector.scoreRisk` — no requirement asked for it.

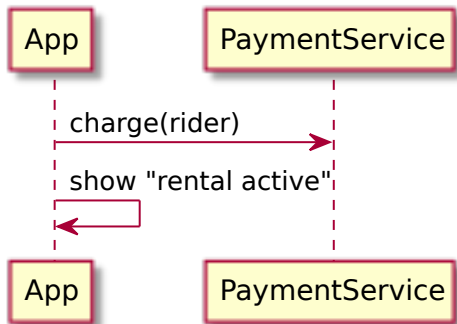
Critique: *“What requirement does this message serve? Who is responsible for it? If nothing, it shouldn't be here.”*

Defect #2: Impossible Order



`rentalStarted` is sent *before* `startRental` requests it. **Critique:**
“Read top to bottom — can step 1 happen before step 2 that causes it?”

Defect #3: Missing Return



App declares the rental active without waiting for `charge` to return.

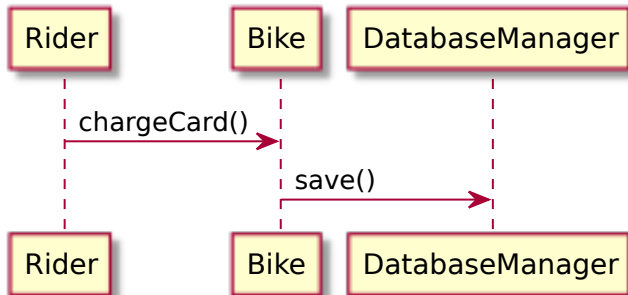
Critique: *"Where's the response to `charge`? How does App know it succeeded?"*

Defect #4: Only the Happy Path

A sequence with no `alt` — payment never declined, bike never already taken, network never down.

Critique: *“What happens when payment fails? AI drew the success case and stopped.”*

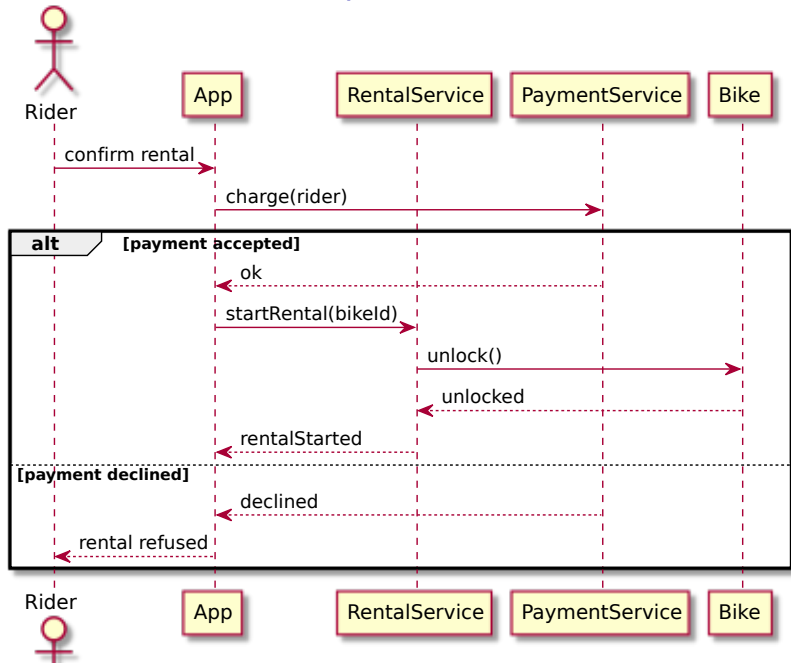
Defect #5: Message to a Stranger



Rider calls `Bike.chargeCard` — but the class diagram has no such association, and `DatabaseManager` is an invented lifeline.

Critique: *“Could these objects even talk? Check the class diagram.”*

The Critic's Corrected Sequence



How to Read an AI Sequence

A fixed order, every time:

1. Does each **message** map to a real responsibility? (or is it invented)
2. Is the **order** causally possible? (no result before its cause)
3. Are the **returns** present? (no dangling calls)
4. Is the **failure path** modelled? (not just the happy path)

The Critique Loop on Behaviour

read -> name the fabricated/missing messages -> re-prompt with the responsibilities + the failure case -> re-read.

Same architect-and-critic loop as Weeks 2-5 — now on an interaction.

The Human Decides What Really Happens

AI can write a plausible conversation between objects forever.
Whether it's the interaction the system *actually* performs — that's the architect-and-critic call.

It's what the oral defense checks, and AI can't make it for you.

This Week's Lab: Lab 3

Lab 3 runs this week — a **team defect hunt on flawed structural artifacts** (class, package, component) from Weeks 4-5. Bring your structural read-orders.

The *behavioural* defect hunt — sequences and the diagrams in Week 7 — is Lab 4 (Week 8).

Next Week: Behavioral II (Week 7)

Sequences show *one* interaction. But an object has a whole **lifecycle**, and a workflow has many steps.

Week 7: state machines (object lifecycles) and activity diagrams (workflows) — and AI's orphan states, unreachable transitions, missed guards.

Critique, Rationale, Traceability — The Through-Line

Today you drilled:

- ▶ **Critique** — read an AI sequence and named its fabricated and missing messages.
- ▶ **Traceability** — the sequence is the behavioural node in the trace: requirement -> use case -> sequence -> class (the structure came in Weeks 4-5).

Rationale (why you directed AI a certain way) lands in your project narrative.

That's It For Today

- ▶ Next lecture (Week 7): behavioral II — state + activity.
- ▶ This week's lab (Lab 3): team defect hunt on flawed structural artifacts.

Questions?